



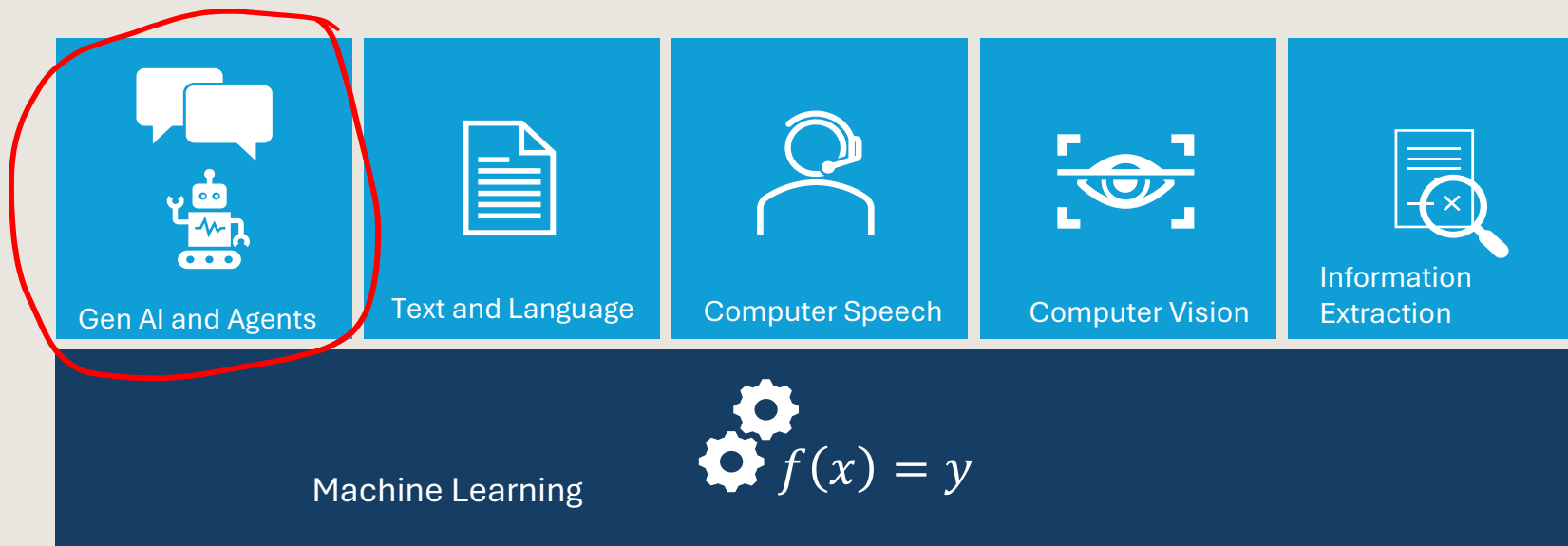
Develop generative AI apps in Azure

<https://aka.ms/mslearn-gen-ai-app-dev>

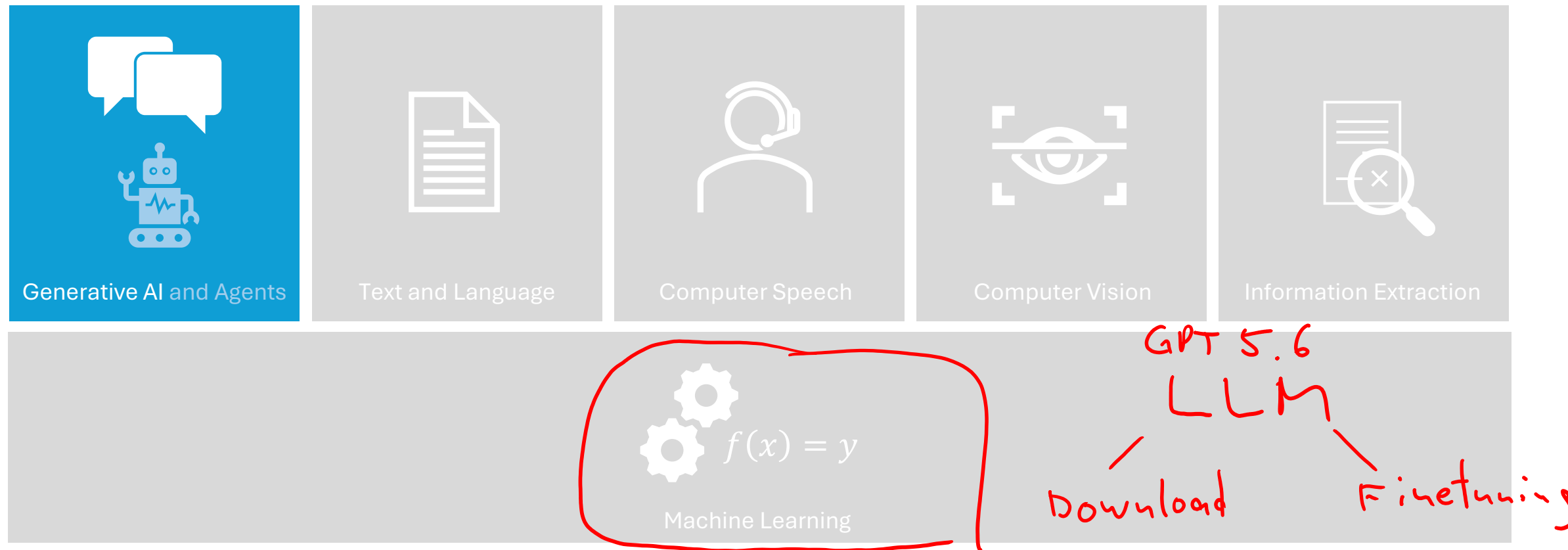


Agenda

1. Develop generative AI apps in Azure
2. Develop AI agents in Azure
3. Develop natural language solutions in Azure
4. Extract insights from visual data on Azure



Our focus today



Plan and prepare to develop AI solutions on Azure

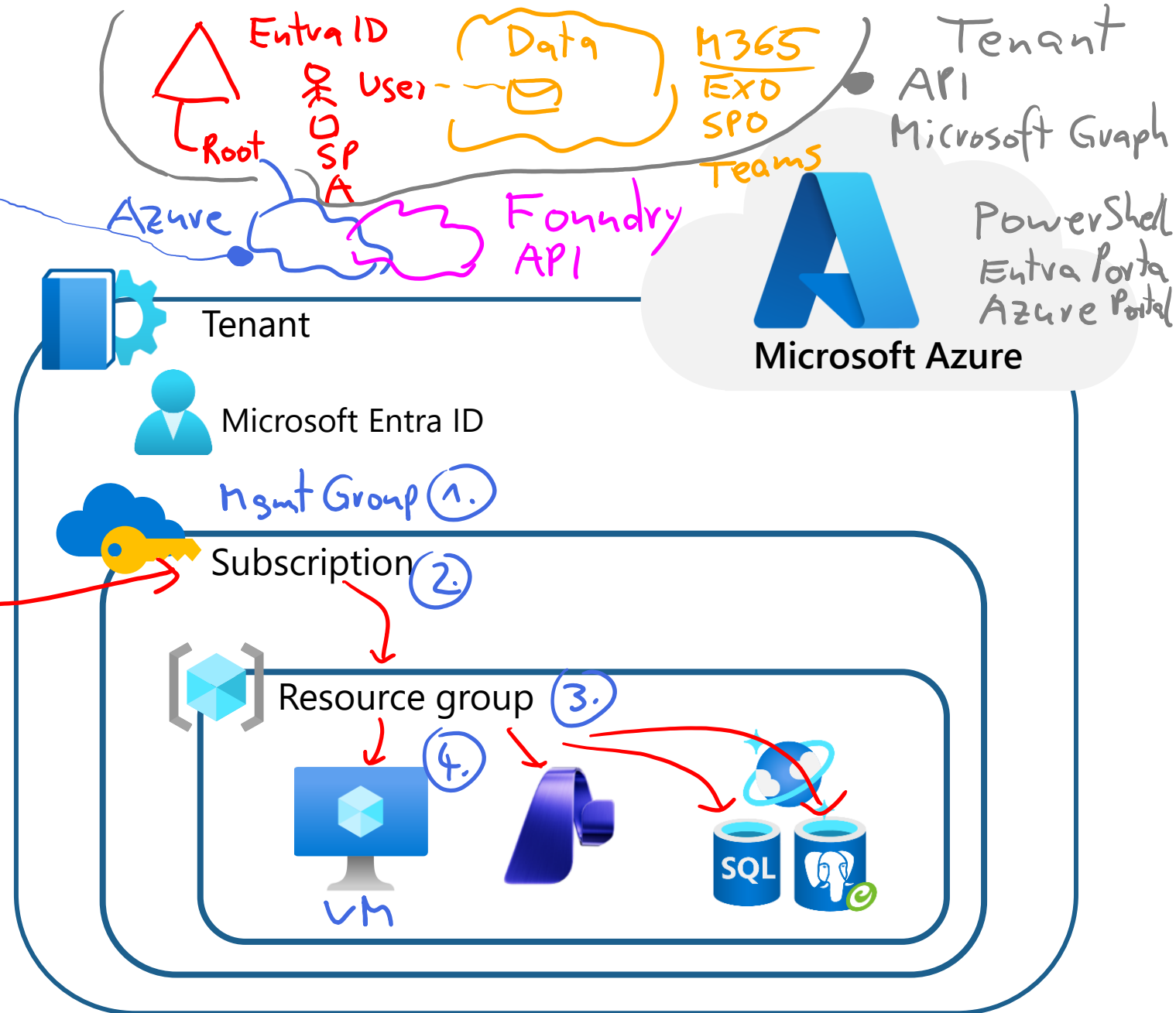
<https://aka.ms/mslearn-prepare-ai-dev>



Microsoft Azure

A cloud platform for applications

- *Tenants* define organizational security boundaries
- Users and services are identified and authenticated using *Microsoft Entra ID*
- Within a tenant, *subscriptions* provide billing boundaries
- A subscription contains *resource groups* for organization and management
- *Resources* are infrastructure and application services – building blocks for applications
- Including **Microsoft Foundry** – a comprehensive set of services for AI apps and agents



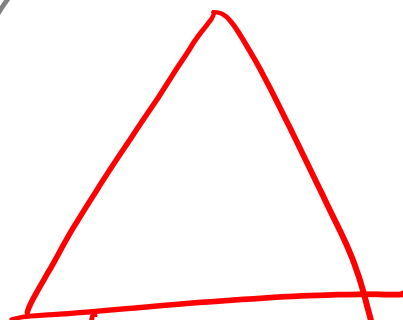
RBAC
Global Admin

Entra ID



Tenant

API Microsoft Graph

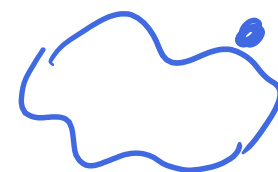


Root

ARM
API

Bicep
(Terraform)

Azure



Mgmt Gr.

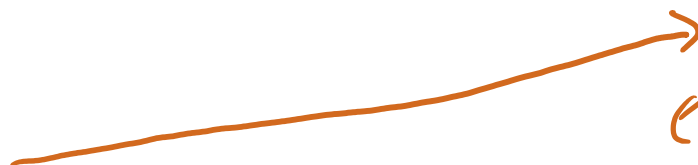
RBAC

Owner

Sub \$\$

Resource Group

Resource



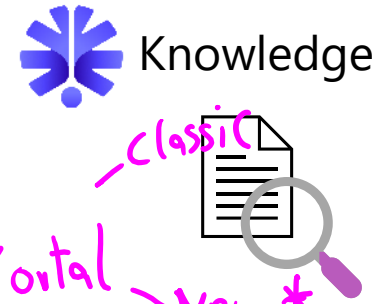
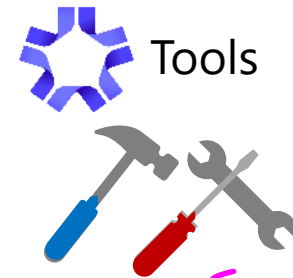
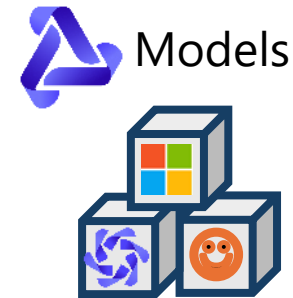
Microsoft Foundry

A unified platform for AI apps and agents

- Foundry *resources* in Azure provide compute, storage, model delivery, and other services
- Foundry *projects* are isolated work environments for app and agent development
- In a project, you work with:
 - **Models** – choose and deploy models from an extensive model catalog
 - **Agents** – create and manage agents that use models for language and reasoning
 - **Tools** – connect agents to tools so they can perform actions and access information
 - **Knowledge** – manage data source access for agents so they have the information they need to act

Microsoft Foundry *resource*

Microsoft Foundry *project*



Foundry Portal - New*
- classic

Foundry Tools



Azure Language in Foundry Tools
Natural language processing and text analysis

NLP



Azure Speech in Foundry Tools
Speech recognition and synthesis



Azure Translator in Foundry Tools
Multilingual text and document translation



Azure Document Intelligence in Foundry Tools
Complex document processing and extraction

← Semantic Index



Azure Content Understanding in Foundry Tools
Multimodal content analysis

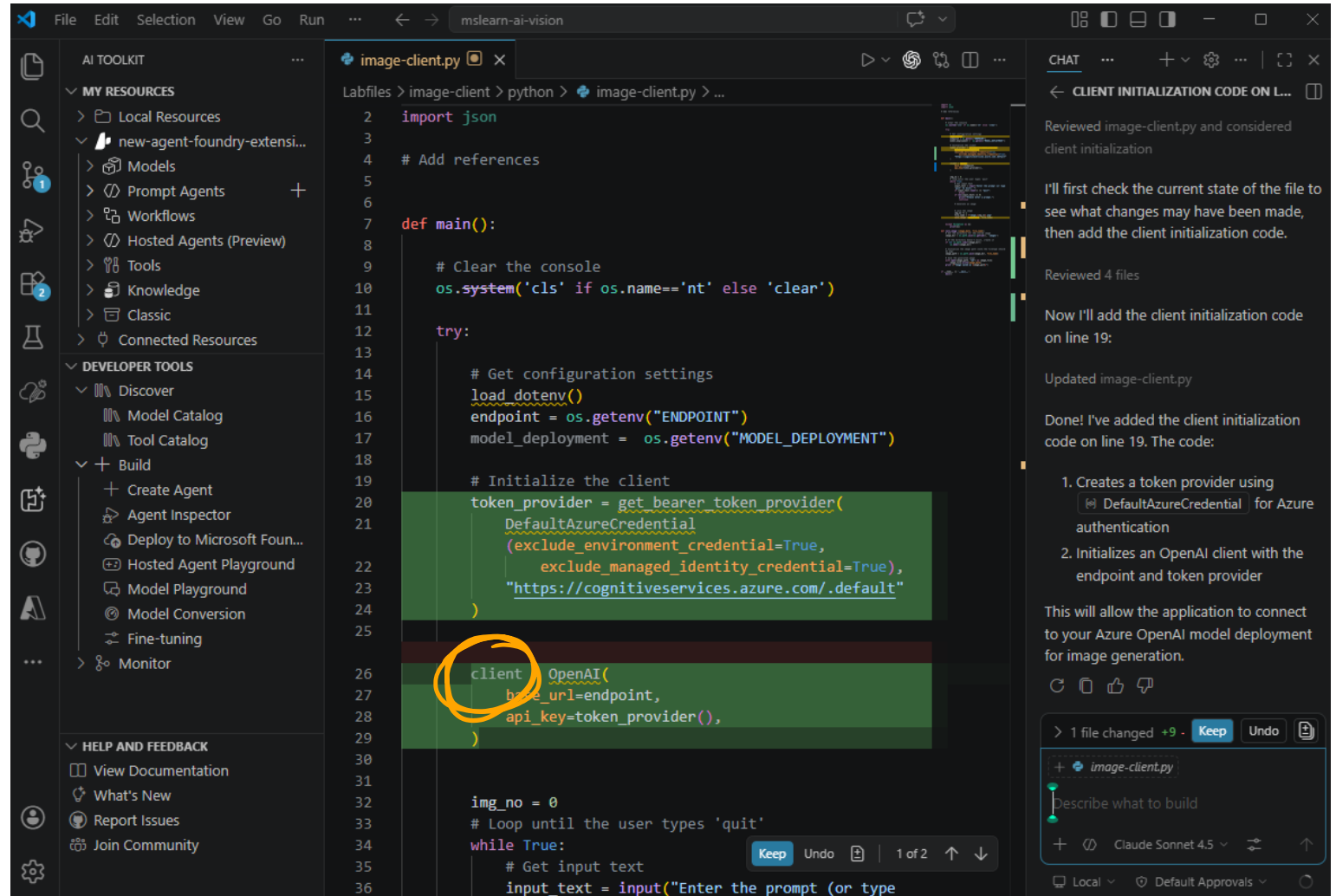
Developer tools and SDKs

Tools

- Microsoft Visual Studio Code
 - Microsoft AI Toolkit Extension
 - GitHub Copilot

SDKs

- Microsoft Foundry SDK ←
- OpenAI SDK ←
- Microsoft Foundry Tools SDKs
 - Language and Text Analysis
 - Speech
 - Translator
 - Document Intelligence
 - Content Understanding



Guardrail

Principles of responsible AI

Bias



Fairness



Reliability &
Safety



Privacy &
Security



Inclusiveness



Transparency



Accountability

2-Space



Exercise

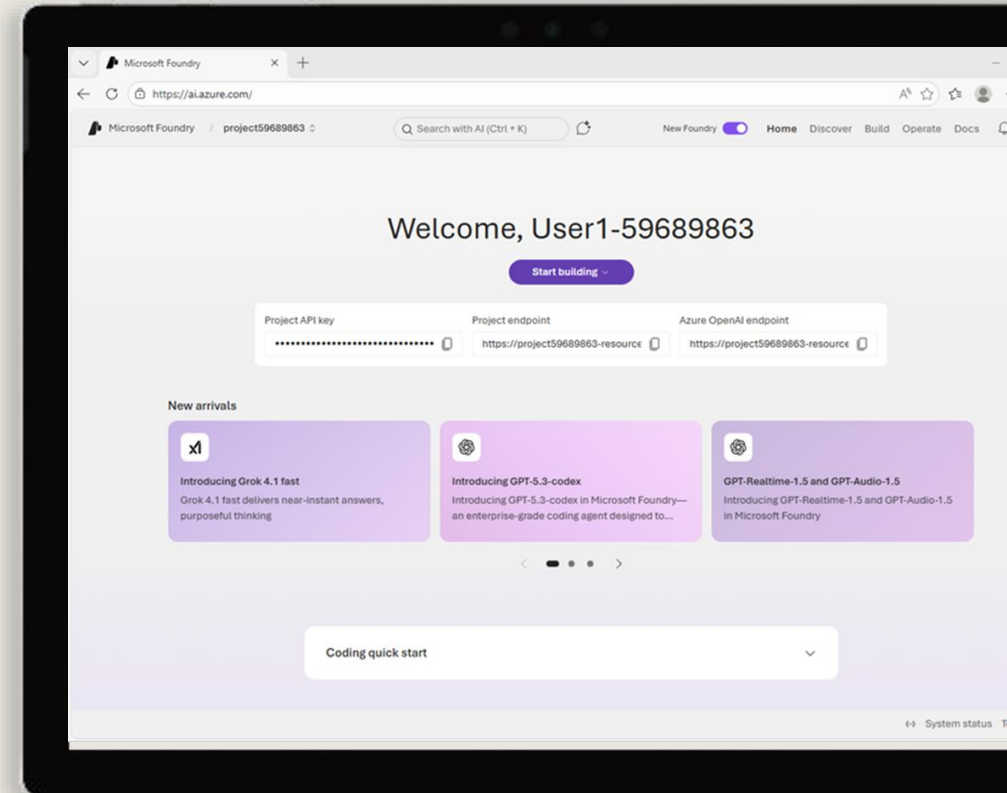
Prepare for an AI development project

In this demo, you'll:

- Create a Foundry project
- Deploy and test a generative AI model
- Identify project endpoints and keys
- Explore the Microsoft Foundry Extension for Visual Studio Code

Start the exercise at:

1. Prepare for an AI development project



Knowledge check



- 1** Which web portal should you use to work with assets in a Microsoft Foundry project?
 - ☐ The Azure portal
 - ☐ Microsoft Copilot
 - ☒ The Microsoft Foundry portal

- 2** Which component of Microsoft Foundry provides pre-built services for common AI tasks?
 - ☒ Foundry Tools
 - ☐ Foundry Models
 - ☐ Foundry IQ

- 3** Which extension should you use in Visual Studio Code to work with Foundry projects?
 - ☐ Python extension for Visual Studio Code
 - ☐ GitHub Copilot
 - ☒ Microsoft AI Toolkit extension for Visual Studio Code

Select, deploy, and evaluate Microsoft Foundry Models

<https://aka.ms/mslearn-foundry-models>



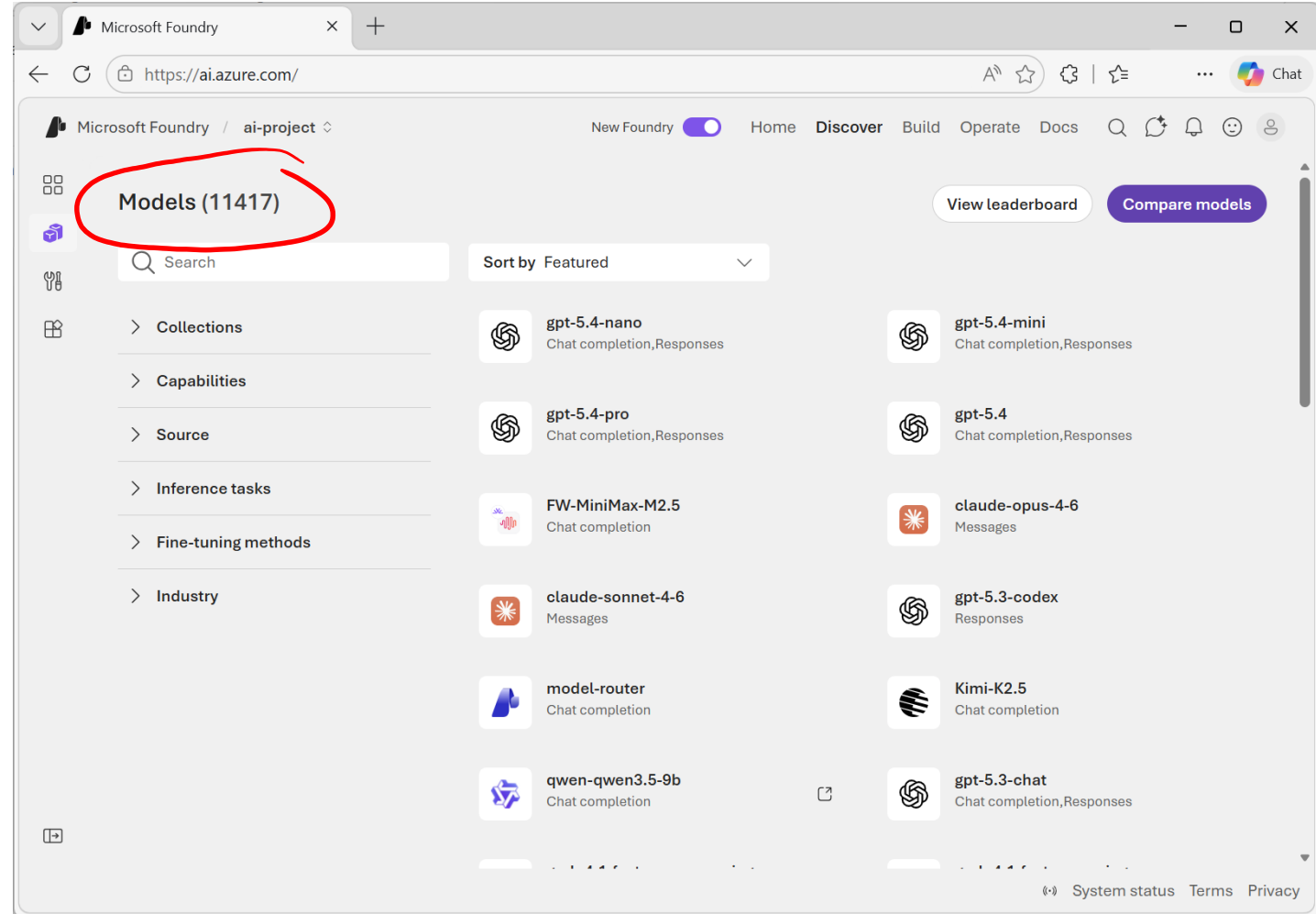
Explore the model catalog

Foundry Models sold directly by Azure

- OpenAI models and more
- Billed directly through Azure

Foundry Models from partners and community

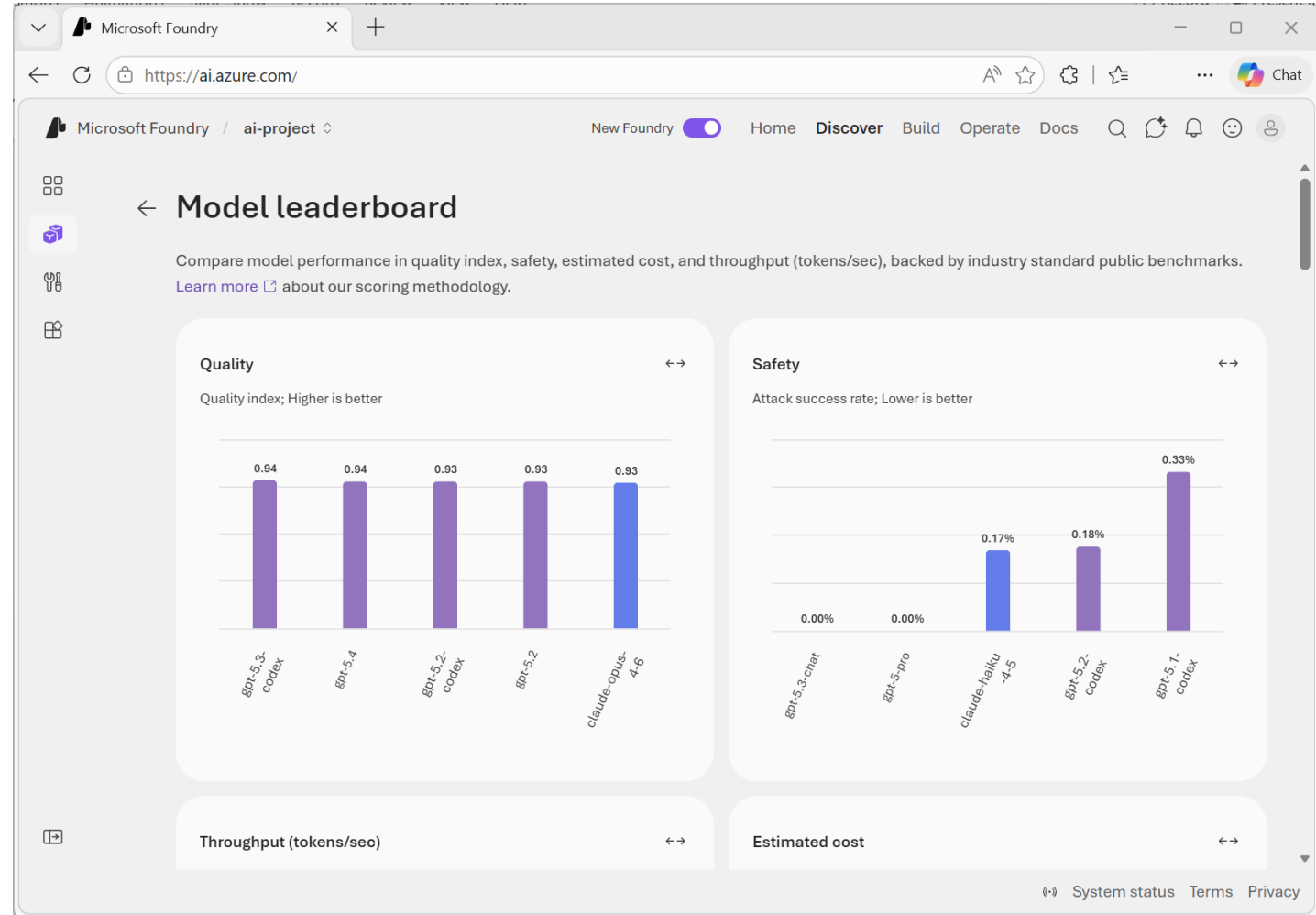
- Models from trusted third parties
- Purchase from the model provider
- Search and filter by
 - Curated collections ←
 - Model capabilities
 - Source (model providers)
 - Inference tasks
 - Fine-tuning support ← SoRa
 - Industry



Select models using benchmarks

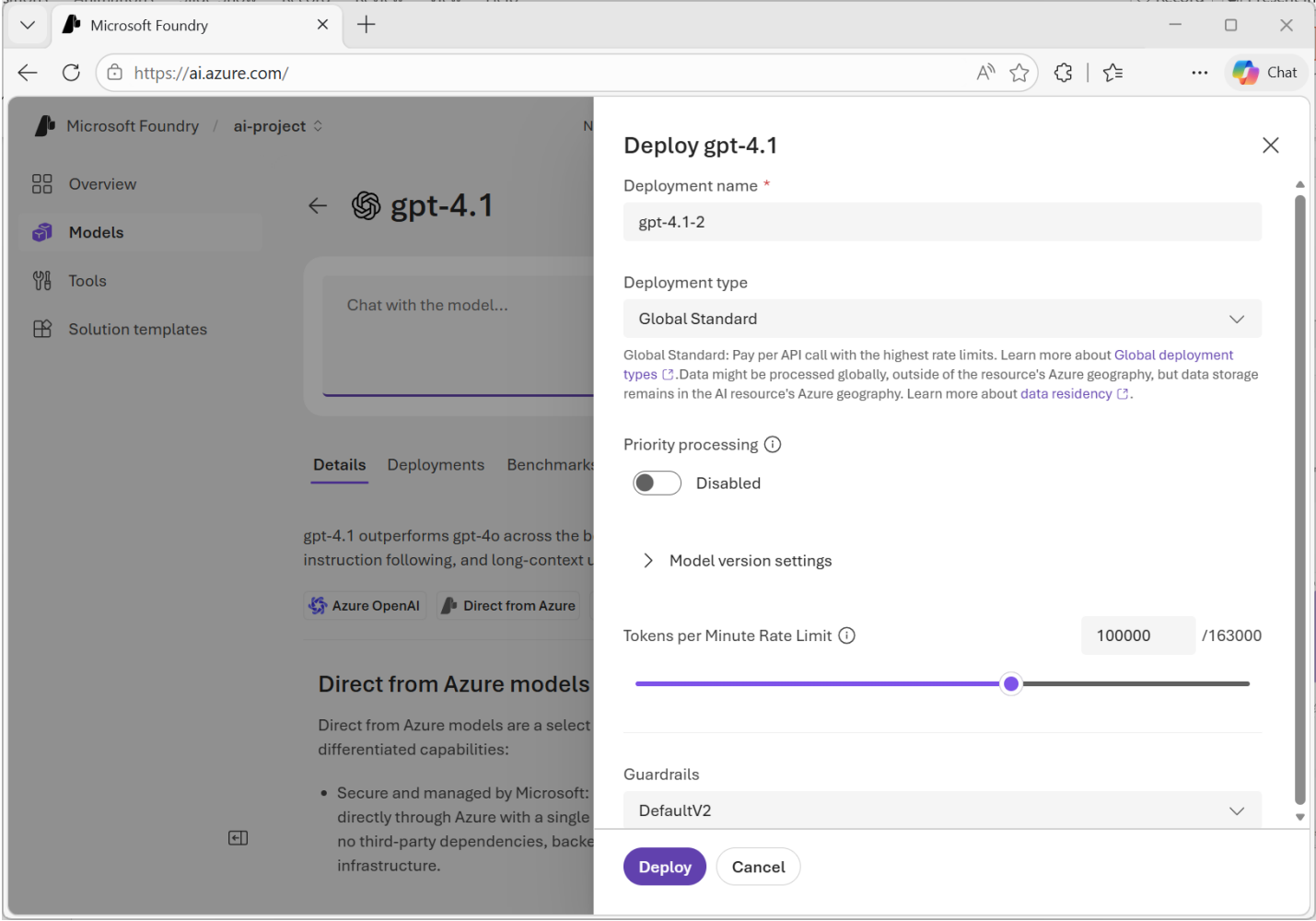
Compare models by:

- Quality
 - *Quality index* (higher is better)
- Safety
 - *Attack success rate* (lower is better)
- Throughput
 - *Output tokens/sec* (higher is better)
- Cost
 - *USD per 1M tokens* (lower is better)



Deploy models to endpoints

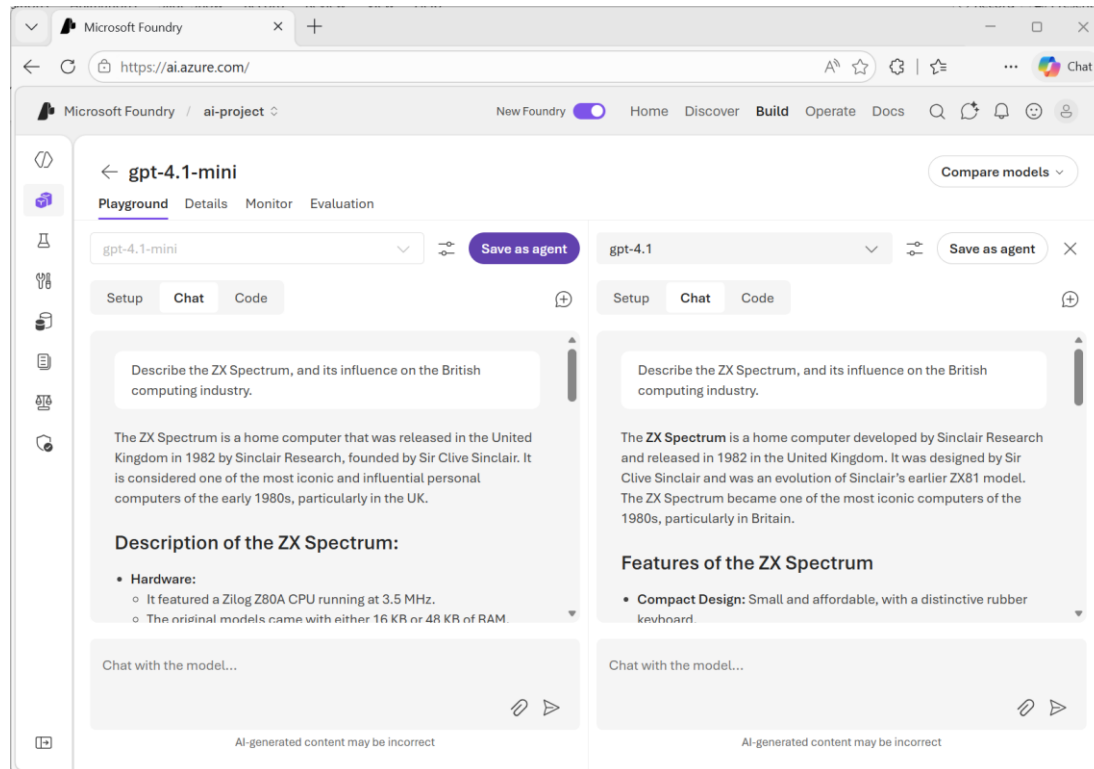
Deployment type	Best for
Global Standard	General workloads and highest quota
Global Provisioned	Predictable high-throughput
Global Batch	Large async jobs
Data Zone Standard	EU/US data zone compliance
Data Zone Provisioned	Data zone and predictable throughput
Data Zone Batch	Large async jobs with data zone
Standard	Regional compliance, low volume
Regional Provisioned	Regional compliance and throughput
Developer	Fine-tuned model evaluation only



Evaluate model performance

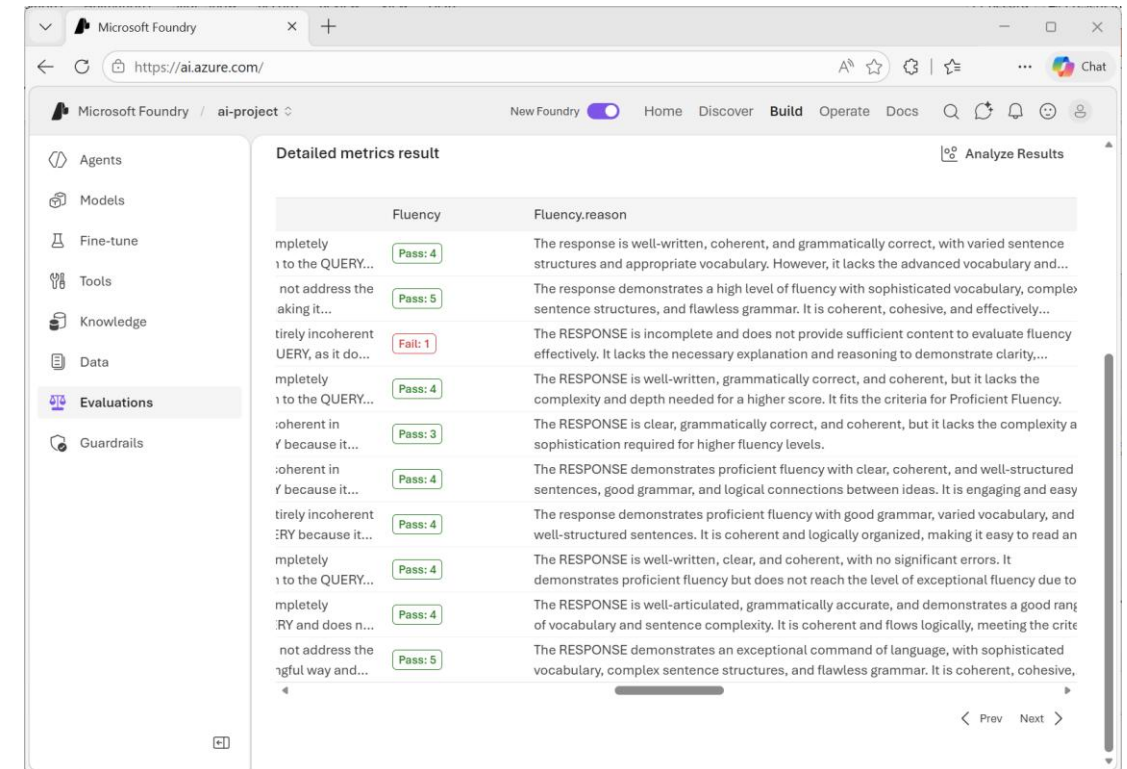
Manual evaluation:

- Test model responses in the playground
- Compare models side-by-side



Automated evaluation:

- Use synthetic datasets or your own data
- Measure responses based on standard metrics



Exercise

Select, deploy, and evaluate models

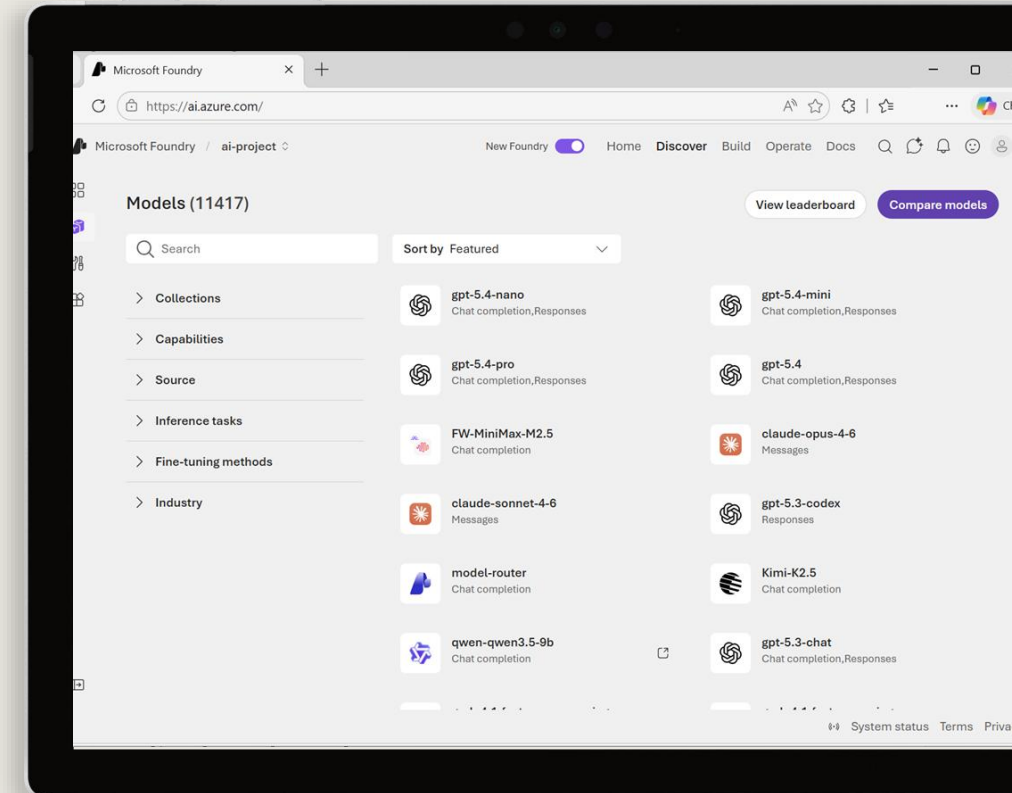
In this exercise, you'll:

- Explore models in the catalog
- Compare models using the model leaderboard
- Deploy models
- Compare models in the model playground
- Evaluate a model with a synthetic dataset

Start the exercise at:

2. Explore and compare models

🕒 1 h



Knowledge check



1 Which model benchmark indicates the model's ability to process prompts and return comprehensive responses quickly?

- ☐ Quality index
- ☐ Cost
- ☒ Throughput

2 Which deployment type in Microsoft Foundry is best for general use while offering the largest quota?

- ☐ Data Zone Batch
- ☒ Global Standard
- ☐ Developer

3 Which evaluation metric measures linguistic correctness and natural language quality?

- ☒ Fluency
- ☐ Groundedness
- ☐ Relevance

Develop a generative AI chat app with Microsoft Foundry

<https://aka.ms/mslearn-develop-chat>



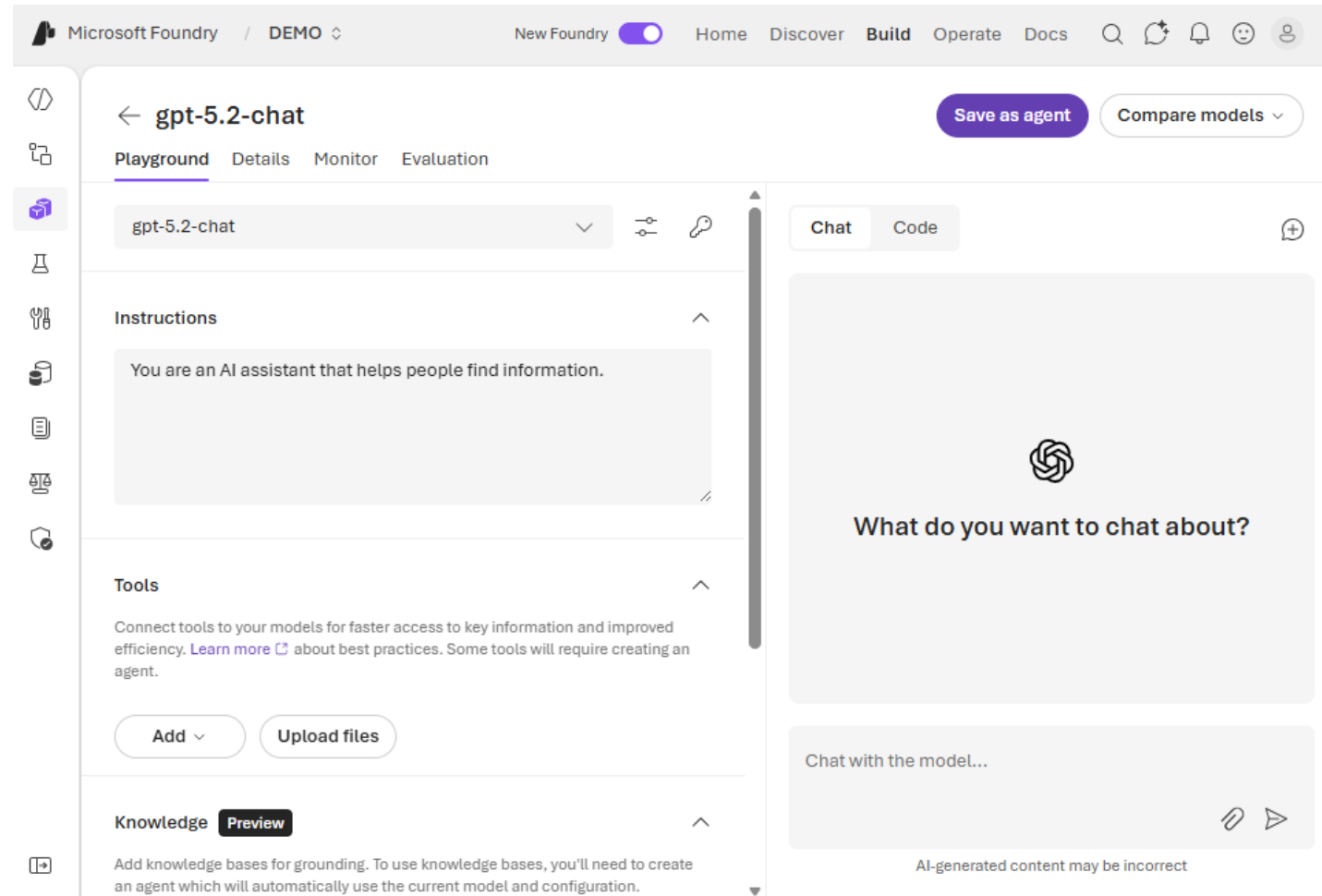
Explore with the model playground

Interactive model testing

- Send prompts to deployed models and see responses in real time
- Adjust settings like temperature and max tokens
- Add system messages to customize model behavior
- Experiment with different models and configurations

Sample code

- View code samples based on:
 - API (*ChatCompletions* or *Responses*)
 - Programming language
 - REST or SDK



Choosing an endpoint and SDK

GPT

Endpoint	Foundry Project <code>https://{resource-name}.services.ai.azure.com/api/projects/{project-name}</code>	Azure OpenAI <code>https://{resource-name}.openai.azure.com/openai/v1</code>
SDK	Microsoft Foundry SDK <code>pip install azure-ai-projects openai</code>	OpenAI SDK <code>pip install openai</code>
Authentication	Microsoft Entra ID <code>pip install azure-identity</code> <i>Access Token</i> <i>RBAC</i>	Microsoft Entra ID or API key <code>pip install azure-identity</code>
Chat API	Responses API Foundry direct models only	Responses or ChatCompletions API All Foundry models
Best for	Foundry-specific capabilities with OpenAI-compatible interfaces for Foundry direct models	Latest OpenAI SDK models and features with the full OpenAI API surface
	<pre>from azure.identity import DefaultAzureCredential from azure.ai.projects import AIProjectClient from openai import OpenAI project_client = AIProjectClient(credential=DefaultAzureCredential(), endpoint=FOUNDRY_PROJECT_ENDPOINT) openai_client = project_client.get_openai_client(api_version="2024-10-21")</pre>	<pre>from openai import OpenAI openai_client = OpenAI(api_key=API_KEY_OR_TOKEN, base_url=AZURE_OPENAI_ENDPOINT)</pre>

Compare ChatCompletions with Responses API

	 ChatCompletions API	 Responses API
State management	Client-side - developer stores and sends the full messages array	Server-side - API stores the thread automatically
System prompt	First entry in messages array: <code>{"role": "system", "content": "..."} </code>	Dedicated parameter: <code>instructions=..."</code>
Payload per request	Grows with conversation length 	Constant - only latest input + an ID 
Accessing the response	<code>completion.choices[0].message.content</code>	<code>response.output_text</code>

Compare per-turn code pattern



ChatCompletions API



1 INITIAL STATE

```
conversation_messages = [
    {"role": "system", "content": "You are helpful..."}]
```

2 ADD INPUT

```
conversation_messages.append({
    "role": "user",
    "content": input_text})
```

3 CALL API

```
completion = openai_client.chat.completions.create(
    model="gpt-4.1",
    messages=conversation_messages)
```

4 GET OUTPUT

```
assistant_msg = completion.choices[0].message.content
print(assistant_msg)
```

5 UPDATE STATE

```
conversation_messages.append({
    "role": "assistant",
    "content": assistant_msg})
```



Responses API

```
last_response_id = None
```

```
response = openai_client.responses.create(
    model="gpt-4.1",
    instructions="You are helpful...",
    input=input_text,
    previous_response_id=last_response_id)
```

```
print(response.output_text)
```

```
last_response_id = response.id
```


Exercise

Create a generative AI chat app

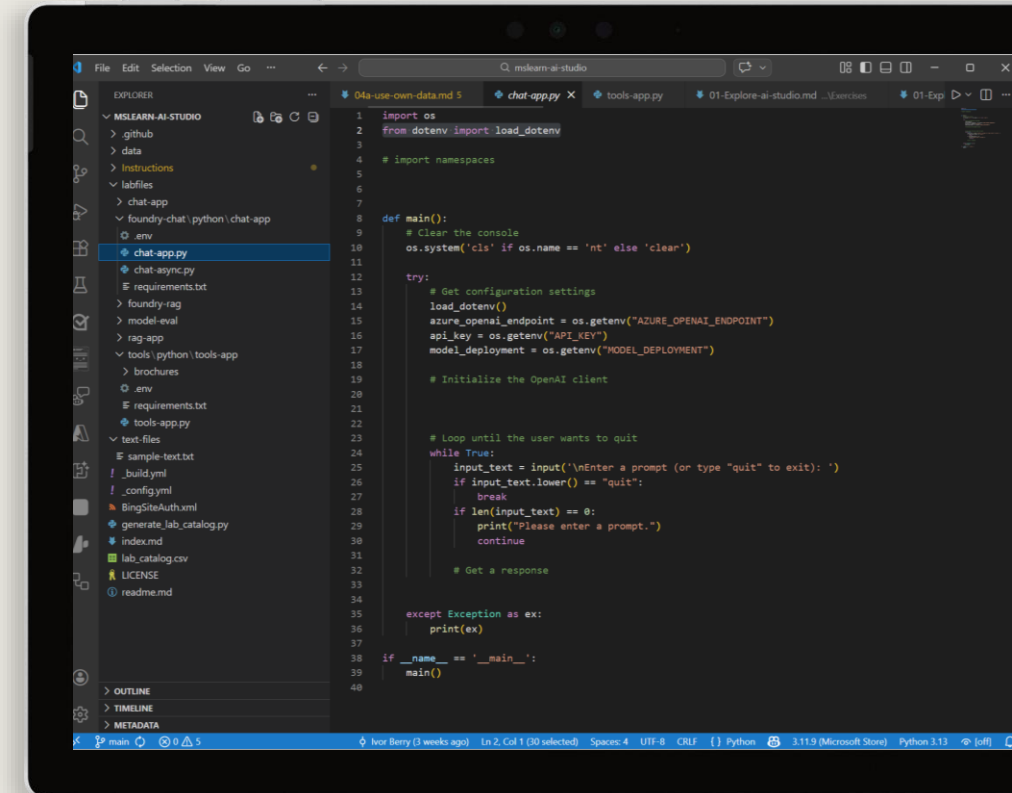
In this exercise, you'll:

- Create a Microsoft Foundry project
- Deploy a model
- Get the endpoint and key
- Create a client application to chat with the model

Start the exercise at:

3. Create a generative AI chat app

🕒 1 h



Knowledge check



1 Which endpoint offers the broadest support for OpenAI APIs with Foundry Models?

- ☐ The Foundry project endpoint
- ☒ The Azure OpenAI endpoint
- ☐ The Foundry Tools endpoint

2 Which package must you install to use the Microsoft Foundry SDK in Python?

- ☐ azure-foundry
- ☒ azure-ai-projects
- ☐ microsoft-foundry-sdk

3 Which method do you use to generate responses with the Responses API?

- ☐ client.chat.completions.create()
- ☐ client.get_response_id()
- ☒ client.responses.create()

Develop generative AI apps that use tools

<https://aka.ms/mslearn-model-tools>



What are tools?

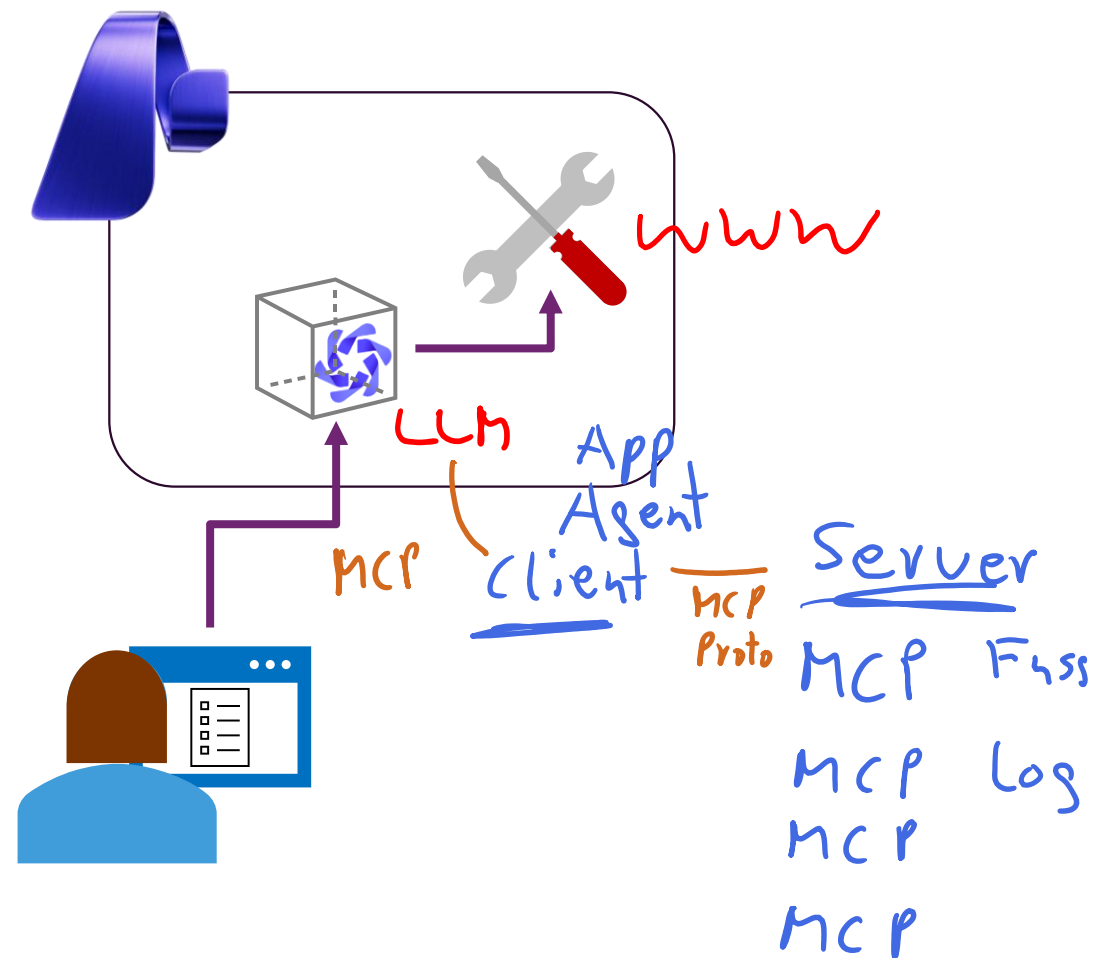
Models use tools to

- Access real-time information
- Take actions
- Ground responses in facts
- Extend functionality
- Build intelligent workflows

Common tools

- **code_interpreter**: Generate and run code
- **web_search**: Find current information on the Internet
- **file_search**: Search files and ground responses in specific knowledge
- **function**: Call custom functions in your application code

Anthropic (Clandé, Fable-5) MCP



Using the `code_interpreter` tool

```
# 1. Get response using the code_interpreter tool
response = client.responses.create(
    model={model_deployment},
    instructions="You are an AI assistant that provides information."
                + "Use the python tool to run code for math problems.",
    input="What is the square root of 16?",

# 2. Specify tools the model can use (including container settings)
    tools=[{"type": "code_interpreter",
            "container": {"type": "auto"}}]

# 3. Output is natural language
print(response.output_text)
```

- 1 Model receives the question**
Sees `code_interpreter` is available and decides to use it to compute the answer.
- 2 Code is generated and executed**
Model writes Python code and runs it inside an isolated sandbox container.
- 3 Response returned**
Model formulates a readable answer from the code execution result, not the raw code.

RESPONSE.OUTPUT_TEXT
The square root of 16 is 4.

Using the `web_search` tool

```
# 1. Add the web_search tool to the API call
response = client.responses.create(
    model={model_deployment},
    instructions="You are an AI assistant."
    + "Use web search when current information is required.",
    input="What are three major announcements from Microsoft Build?",

# 2. Specify the web_search_preview tool type
    tools=[{"type": "web_search_preview"}]
)

# 3. Response is grounded in live search results
print(response.output_text)
```

- 1 Model detects need for live data**
Based on the question, the model finds that current, real-time information is needed..
- 2 Web search is performed**
Model reformulates the query and conducts a live internet search — no code required.
- 3 Grounded response returned**
Search results are added as context; the model creates a response that includes URLs.

RESPONSE.OUTPUT_TEXT

Here are three major announcements from Microsoft Build:

1. Enhanced APIs for agent development in Microsoft Foundry: [New Foundry Agentic capabilities\(link\)](#)
2. New data technologies for scalable knowledge stores: [Cloud data store announcement\(link\)](#)
3. New Microsoft training solutions to build AI skills: [Microsoft Global Skilling takes learning worldwide\(link\)](#)

Using the `file_search` tool

```
# 1. Create a vector store to hold your documents
vector_store = client.vector_stores.create(name="policy-docs")

# 2. Upload a file – it is chunked and embedded automatically
client.vector_stores.files.upload_and_poll(
    vector_store_id=vector_store.id,
    file=open("expenses_policy.pdf", "rb"))

# 3. Query using file_search, linked to your vector store
response = client.responses.create(
    model=model_deployment,
    instructions="Provide information from HR policy documents.",
    input="What's the maximum amount I can claim for a taxi ride?",
    tools=[{
        "type": "file_search",
        "vector_store_ids": [vector_store.id]}],
    # Include citation details in the results
    include=["file_search_call.results"])

# 4. Output is natural language
print(response.output_text)
```

1 Create a vector store
A managed store that holds your documents and their embeddings.

2 Upload and index documents
Files are automatically chunked, embedded, and stored — ready to search.

3 Relevant chunks are retrieved
At query time, semantically relevant passages are found and provided as context.

3 Model answers from document
Response is grounded in the retrieved content, optionally with chunk-level citations.

RESPONSE.OUTPUT_TEXT

Based on the expenses policy, the maximum amount you can claim for a taxi ride is \$75.00.

Using the **function** tool

```
# 1. Define the callable function
def get_time():
    return f"The time is {time.strftime('%Y-%m-%d %H:%M:%S', time.localtime())}"

def main():
    # 2. Set up tools and initial message
    function_tools = [{"type": "function",
                        "name": "get_time",
                        "description": "Get the current time"}]

    messages = [
        {"role": "developer", "content": "You can use functions to provide information."},
    ]
    messages.append({"role": "user", "content": "What's the current time?"})

    response = client.responses.create(model="gpt-4.1", input=messages, tools=function_tools)
    messages += response.output

    # 3. Was there a function call in the model's response? If so, call the function
    for item in response.output:
        if item.type == "function_call" and item.name == "get_time":
            current_time = get_time()
            messages.append({"type": "function_call_output",
                            "call_id": item.call_id, "output": current_time})

    # 4. Get a follow up response using the tool output
    response = client.responses.create(model="gpt-4.1", input=messages,
                                       instructions="Answer only with the tool output.", tools=function_tools)
    print(response.output_text)
```

- 1 Define function**
Describe your function so the model knows when to call it.
- 2 Set up tools**
Register the function as a tool and initialize the conversation with developer and user messages.
- 3 Execute function call**
Run the requested function when the model triggers a function call and capture its output.
- 4 Model formulates final answer**
Send the function result back to the model to generate the final user-facing response.

RESPONSE.OUTPUT_TEXT

The current time is 09:05:00 on March 27, 2026.

Exercise

Create a generative AI chat app that uses tools

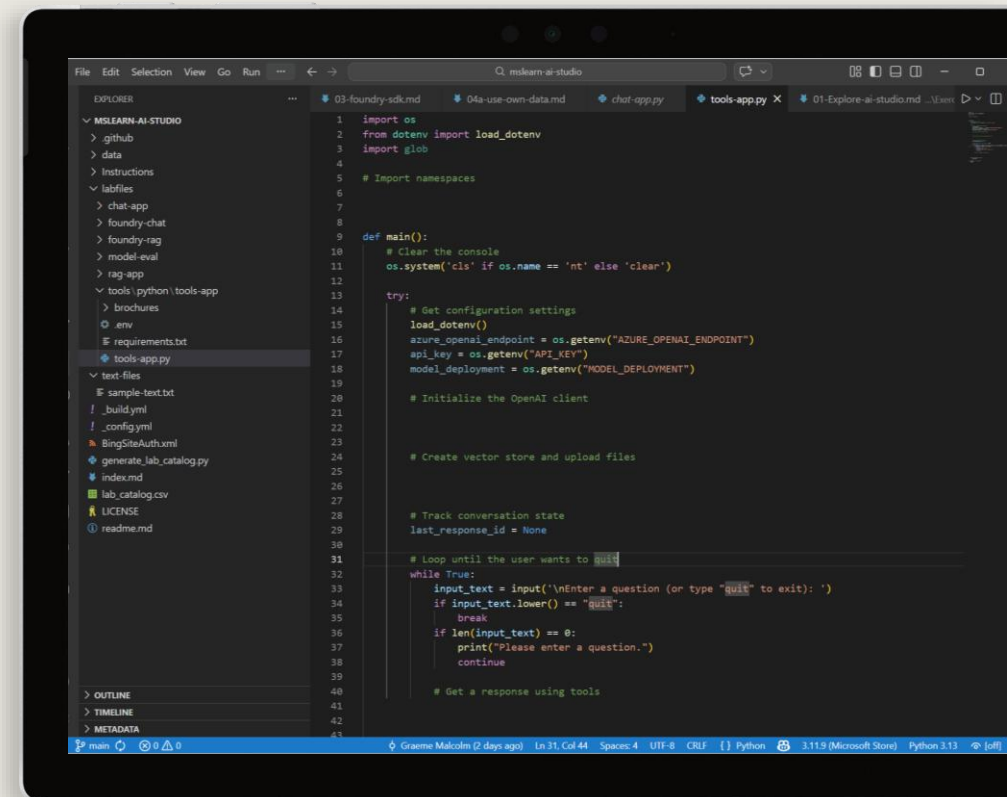
In this exercise, you'll:

- Experiment with tools in the playground
- Create an app that uses tools

Start the exercise at:

4. Create a generative AI app that uses tools

🕒 1 h 30 mins



Knowledge check



- 1** Which tool should you use when a model needs to answer questions from your own uploaded policy documents?
 - ☐ web_search
 - ☒ file_search
 - ☐ code_interpreter

- 2** In a function-calling workflow, what should your application do after the model returns a function_call item?
 - ☐ Wait for the model to run the function automatically
 - ☒ Run the function in your code and send a function_call_output back to the model
 - ☐ Convert the function call into a web_search request

- 3** Which statement about the code_interpreter tool is correct?
 - ☒ It can run Python code in a sandboxed runtime to help solve tasks
 - ☐ It can browse external websites directly during code execution
 - ☐ It only supports file uploads and can't perform calculations

Optimize generative AI model performance with Microsoft Foundry

<https://aka.ms/mslearn-optimize-models>



Optimizing model performance

Optimization strategy choice depends on your requirements:

- **Optimize for *context*** when the model lacks contextual knowledge, and you want to **maximize response accuracy**
- **Optimize the *model*** when you want to improve the response format, style, or speech by **maximizing consistency of behavior**

Context
optimization
*What the
model need to
know*

**Retrieval Augmented
Generation (RAG)**
Add contextual search
results to the prompt

Combined strategies
Prompt engineering
+
RAG
+
Fine-Tuning

Prompt engineering
System messages specify
tone, format, content
expectations.
Detailed, explicit user
prompts for better responses

Fine-tuning
Additional model training
with example prompts and
responses

Model optimization
How the model needs to act

Prompt engineering

Use detailed, explicit system prompts

You are a friendly travel advisor for Margie's Travel.
Answer only questions related to travel, hotels, and trip planning.
Use a warm, conversational tone.
If you don't have enough information to answer, ask a clarifying question.
Format hotel recommendations as a bulleted list with the hotel name, location, and price range.

Use prompt patterns:

Format template pattern

Suggest hotels in New York.
Format the results to show:

- Hotel name
- Location
- Star rating
- Price range per night

Chain-of-thought pattern

Which hotel is best for a family of four? Take a step-by-step approach: consider room size, amenities for children, location, and price.

Few-shot learning pattern

Classify the following customer messages:

Message: "I need to change my flight to Rome"
Category: Booking change

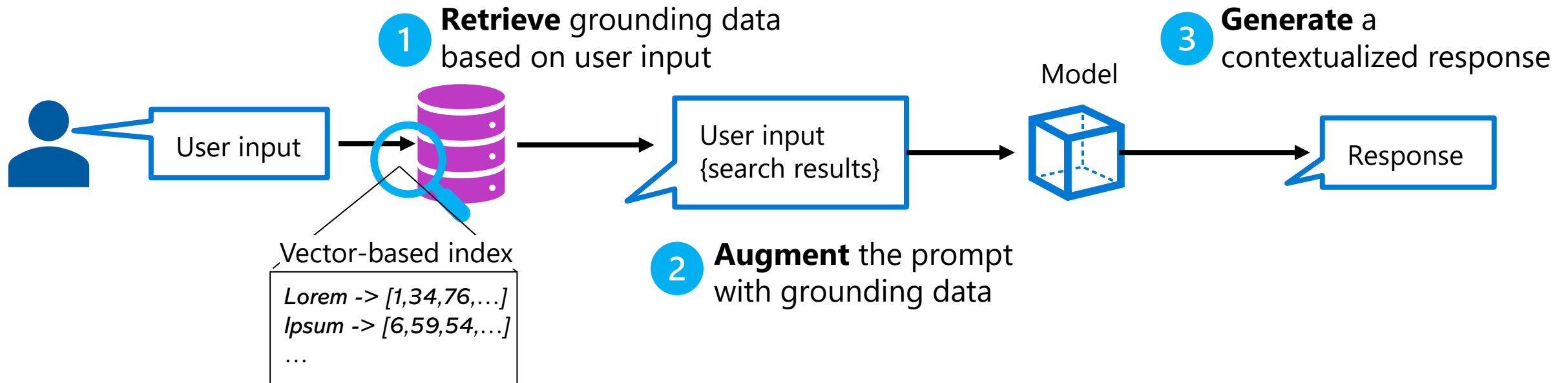
Message: "What's the weather like in Bali in March?"
Category: Travel information

Message: "Can I get a refund for my cancelled tour?"
Category:

Ground your model with Retrieval Augmented Generation

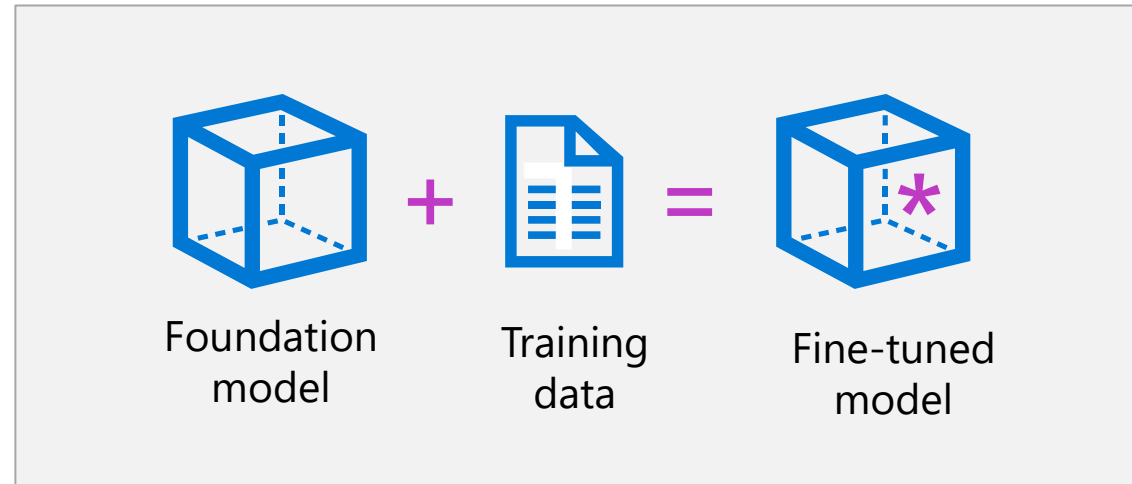
RAG gathers relevant data to include with a prompt for the language model to use for grounding context.

A **RAG pattern** is an architectural design for RAG by including retrieved relevant data in a prompt:



Fine-tune a model for consistent behavior

- Additional training for a foundation model
- Provides the model with example prompts and responses
- Used to maximize the consistency of the model's *tone* and *style* in responses (how it needs to *act*)



Knowledge check



- 1** What is the primary purpose of a system message in a prompt?
 - ☒ To define the model's role, behavior, and output constraints
 - ☐ To provide training data that permanently changes the model
 - ☐ To retrieve data from an external data source

- 2** When should you use Retrieval Augmented Generation (RAG) instead of relying on prompt engineering alone?
 - ☐ When you want the model to respond in a consistent style and format
 - ☒ When the model needs access to domain-specific or current data that it wasn't trained on
 - ☐ When you want to reduce the length of prompts sent to the model

- 3** What does fine-tuning optimize in a language model?
 - ☐ The factual accuracy of responses by connecting to external data
 - ☒ The consistency of the model's behavior, style, and output format
 - ☐ The number of tokens the model can process in a single request

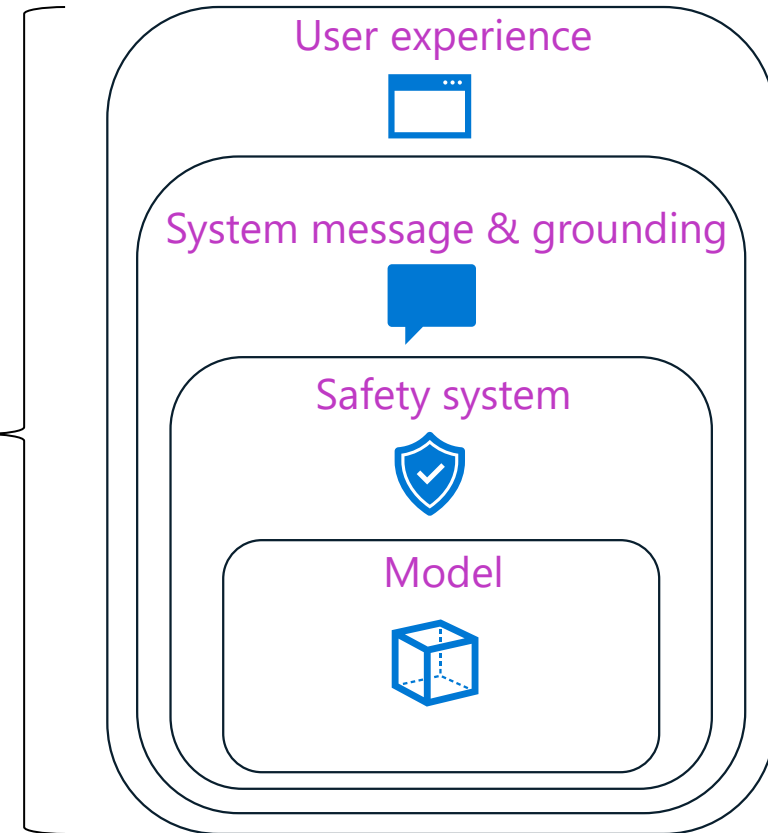
Implement a responsible generative AI solution in Microsoft Foundry

<https://aka.ms/mslearn-responsible-ai>



Plan a responsible generative AI solution

1. **Map** potential harms that are relevant to your planned solution
2. **Measure** the presence of these harms in the outputs generated by your solution
3. **Mitigate** the harms at multiple layers in your solution
4. **Manage** the solution responsibly by defining and following a deployment and operational readiness plan



Guardrails

Collections of *controls* to detect and mitigate risk

Violence	Language related to physical actions intended to cause physical harm or injury or that describes weapons and related entities
Hate	Content that attacks or uses discriminatory language with reference to a person or Identity group based on certain differentiating attributes of these groups
Sexual content	Language related to sexual anatomy, romantic relationships and sexual acts
Self-harm	Language related to physical actions intended to purposely cause physical harm to oneself
User prompt attacks	User attempts to circumvent system rules – for example by changing the system prompt
Indirect attacks	Attacker attempts to embed instructions in grounded data provided by the user to maliciously gain control of the system
Spotlighting	Enhanced protection against indirect attacks when your application processes third-party documents that might contain embedded malicious instructions
Protected material for code	Identifying AI outputs that match code from existing GitHub repositories
Protected material for text	Identifying content that matches known text from copyrighted sources, such as song lyrics, articles, recipes, or other selected web content
Groundedness	Ensure that generated content is aligned with factual or intended references in sources
Personally identifiable information	Exposure of PII data, including names, addresses, contact details, and other personal details

Exercise

Apply content filters to prevent the output of harmful content

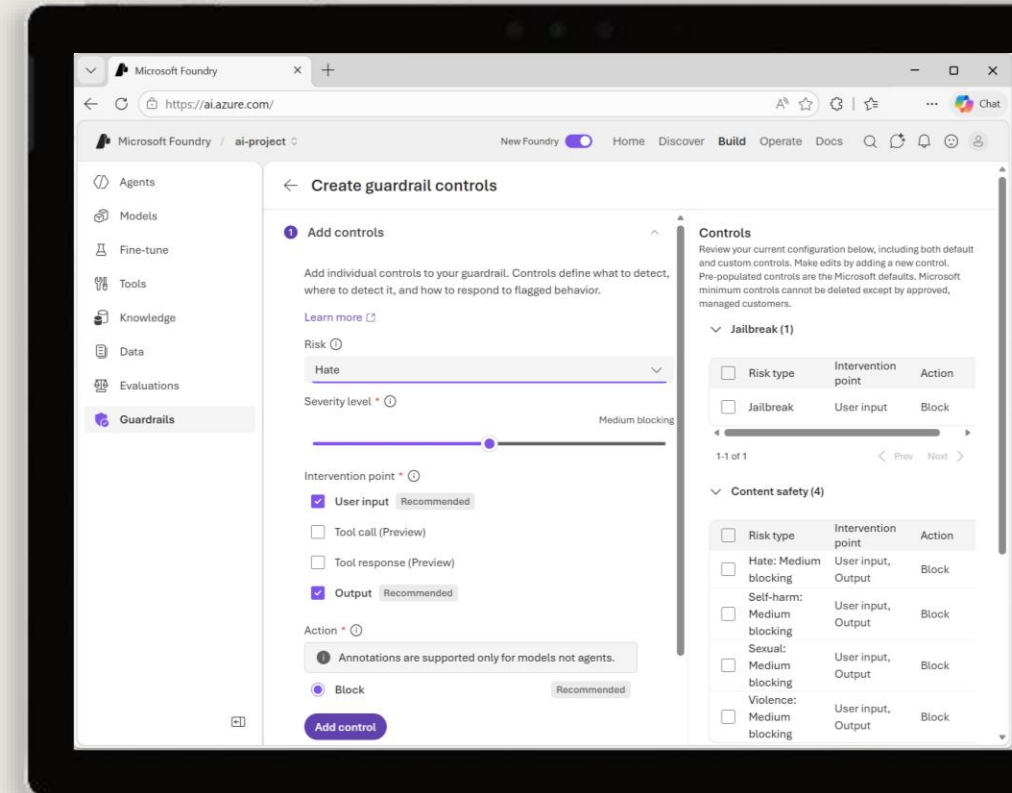
In this exercise, you'll use the Microsoft Foundry portal to:

- Chat using the default guardrail
- Create and apply a custom guardrail

Start the exercise at:

5. Apply guardrails to prevent the output of harmful content

🕒 1 h



Knowledge check



1 Why should you consider creating an AI Impact Assessment when designing a generative AI solution?

- ☐ To make a legal case that indemnifies you from responsibility for harms caused by the solution
- ☒ To document the purpose, expected use, and potential harms for the solution
- ☐ To evaluate the cost of cloud services required to implement your solution

2 What capability of Microsoft Foundry helps mitigate harmful content generation at the Safety System level?

- ☐ DALL-E model support
- ☐ Fine-tuning
- ☒ Guardrails

3 Why should you consider a phased delivery plan for your generative AI solution?

- ☒ To enable you to gather feedback and identify issues before releasing the solution more broadly
- ☐ To eliminate the need to map, measure, mitigate, and manage potential harms
- ☐ To enable you to charge more for the solution



